

Software Defined Ultrasound with GPU Processing

Advanced GPU Short Course

Piotr Jarosik, Marcin Lewandowski, Piotr Karwat

us4us Ltd.

Agenda

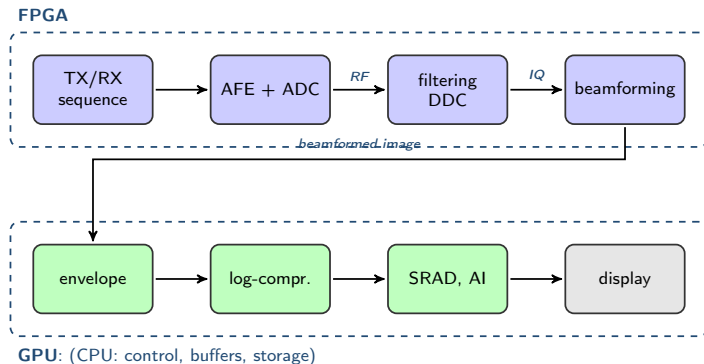
System architecture

Ultrasound

GPU processing

System architecture

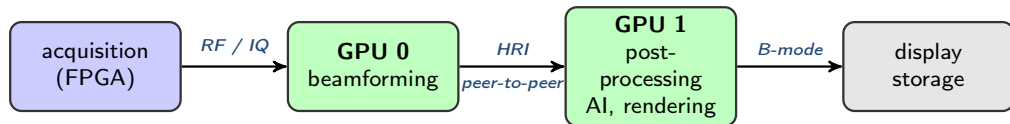
Example: ultrasound imaging system



- **FPGA:** TX/RX sequence execution, RF/IQ acquisition, filtering, beamforming – deterministic, close to the converters.
- **GPU:** everything downstream – flexible, high throughput.

What about the flexibility of RF/IQ data pre-processing and beamforming? β Software Defined Ultrasound and Beamforming

Example: two GPUs



- one device dedicated to the **fixed-cost** part of the pipeline (beamforming), another one to the **variable-cost** part (AI, rendering, recording),
- the frames are passed between the GPUs directly (peer-to-peer),
- the alternative to two devices: partition a single GPU (see: GPU processing).

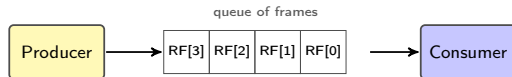
Performance requirements: real-time processing

- non real-time: there are requirements regarding frame rate or PRF
 - ▶ Examples: offline reconstruction, research processing
- hard real-time: a missed deadline is a mission failure which may result in the patient dead, etc.
 - ▶ Example: surgical ultrasound
 - ▶ Requirement: the TX/RX sequence must be executed with constant PRF, the acquisition and processing must keep up with acquisition.
- soft real-time: missed deadline **degrades the quality**, but it is not mission critical
 - ▶ Example Vascular imaging, Color Doppler
 - ▶ Requirement: a dropped or late B-mode frame is acceptable as long as it happens rarely

The barrier between the devices

Every boundary between two devices is a **buffer**: a queue of frames, written by the producer and read by the consumer.

- it decouples the two sides: only the *average* rates have to match,
- at least **two elements** are needed, so that both sides can work at the same time,
- deeper queue: more tolerance for jitter, but more **latency**.



The barrier: what if the consumer is too slow?

block the producer

The producer waits for a free buffer element. Nothing is lost, but the acquisition rate follows the processing $\Rightarrow$ jitter in the TX/RX sequence execution.

drop frames

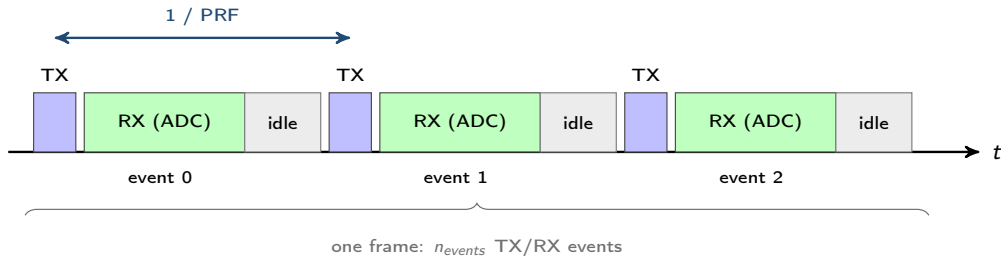
The producer overwrites the oldest or skips the newest element.

error on override

The producer reports an error when it would overwrite an element that is still in use: a hard real-time regime, should be used when no frame may be lost.

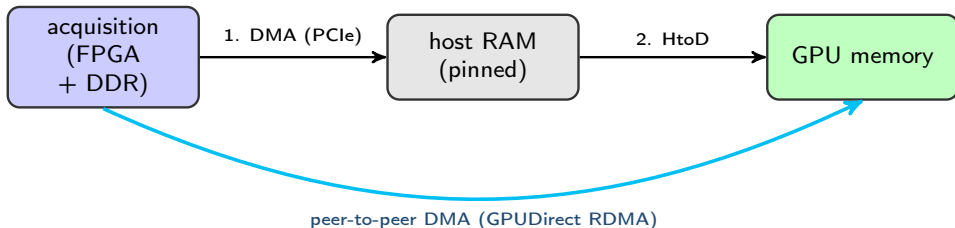
Ultrasound

TX/RX sequence and data acquisition



- the sequence is executed by the **hardware** (FPGA), not by the host: the timing does not depend on the operating system,
- one frame = $n_{\text{events}} \times n_{\text{channels}} \times n_{\text{samples}}$ samples – this is what has to be transferred every frame.

Data path: DMA to the host or straight to the GPU



- classic path: two PCIe transfers and a copy in host RAM,
- **GPUDirect RDMA (NVIDIA) / DMA-BUF, PeerDirect (AMD)**: the acquisition device writes directly into GPU memory – the host RAM is skipped, the CPU only sets the transfer up,
- requires: a device that can do peer-to-peer PCIe, large BAR, driver support (`nvidia-peermem` / `dma-buf`).

GPU processing

Invoking the kernels when new data arrives

The processing is **data-driven**: a new frame arrives, the pipeline runs.

- the acquisition signals a new frame (interrupt, callback, event),
- the host **enqueues** the whole pipeline and returns – it does not wait for the result,
- the same sequence of kernels is executed for every frame.

How to express and control that sequence?

- **CUDA streams**: the queue of operations (+ events between streams),
- **CUDA graphs**: the whole pipeline recorded once, launched per frame,
- **Holoscan SDK**: the pipeline as a graph of operators, with a scheduler.

CUDA Stream: introduction

We can run a sequence of GPU kernels to implement the complete e.g. B-mode imaging:

```
# HtoD
rf_gpu.set(rf_cpu)
iq = ddc_kernel(rf_gpu)
hri = delay_and_sum_kernel(iq)
envelope = envelope_kernel(hri)
bmode = log_compression_kernel(envelope)
# DtoH
bmode_cpu = bmode.get()
```

CUDA Streams: introduction

Refresher: asynchronous vs synchronous operations:

- kernel invocation
 - ▶ **asynchronous**
 - ▶ we have no guarantee that the kernel execution will be finished (or even started) by the next line of our script/host application code
- data transfers (DtoH, HtoD, DtoD): can be **synchronous** or **asynchronous** (it's up to the programmer)

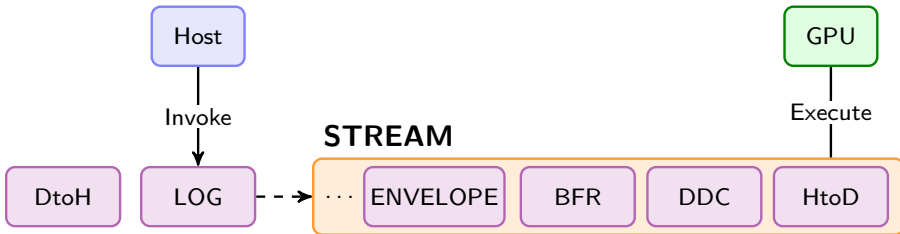
CUDA Streams: example

Refresher:

- host code puts GPU operations into **CUDA Streams**
- **CUDA Streams** → a *queue* (FIFO) of operations
- To wait for all the GPU kernels in the stream to finish: call 'synchronize' on the stream.

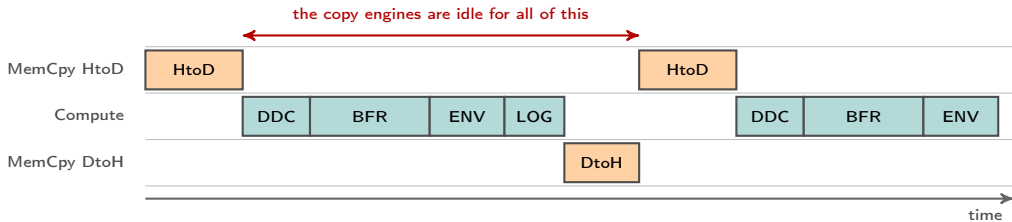
CUDA Streams: example

```
rf_gpu.set(rf_cpu)    # HtoD  
iq = ddc_kernel(rf_gpu) # DDC  
hri = delay_and_sum_kernel(iq) # BFR  
envelope = envelope_kernel(hri) # ENVELOPE  
bmode = log_compression_kernel(envelope) # LOG  
bmode_cpu = bmode.get() # DtoH
```



CUDA Streams: example

```
rf_gpu.set(rf_cpu)    # HtoD
iq = ddc_kernel(rf_gpu) # DDC
hri = delay_and_sum_kernel(iq) # BFR
envelope = envelope_kernel(hri) # ENVELOPE
bmode = log_compression_kernel(envelope) # LOG
bmode_cpu = bmode.get() # DtoH
```



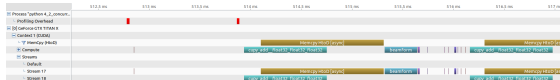
CUDA Streams: multiple streams

Is it possible to run GPU operations in parallel?

- Generally, it's possible, by using multiple streams.
- In practice: it depends what operations we would like to run in parallel:
 - ▶ executing GPU kernels simultaneously – not always guaranteed (it depends on the GPU occupancy, kernel scheduler implementation, etc.)
 - ▶ GPU kernels and data transfers can be overlapped – data transfers can be handled by a separate GPU component **DMA**, that can run in parallel with SMs

CUDA Streams: overlapping data transfer with kernels

```
stream_17 = cp.cuda.Stream(non_blocking=True)
stream_18 = cp.cuda.Stream(non_blocking=True)
with stream_17:
    gpu_array.set(rf)
    img      = beamformer.process(gpu_array)
    envelope = to_envelope.process(img)
    bmode    = to_bmode.process(envelope)
with stream_18:
    a+b
```



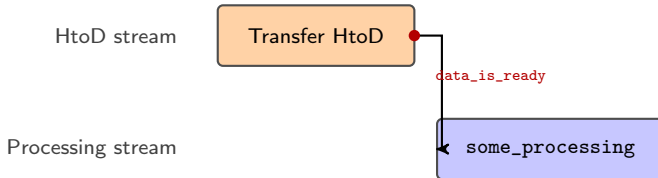
CUDA Streams synchronization: Events

How to make sure, that GPU kernel will not start until the HtoD transfer is finished?

- We can use CUDA **Events**.
- **CUDA event** object represents an event in the CUDA stream execution timeline. An example of such an event is the completion of a GPU kernel, or that the data transfer from host to GPU has finished.

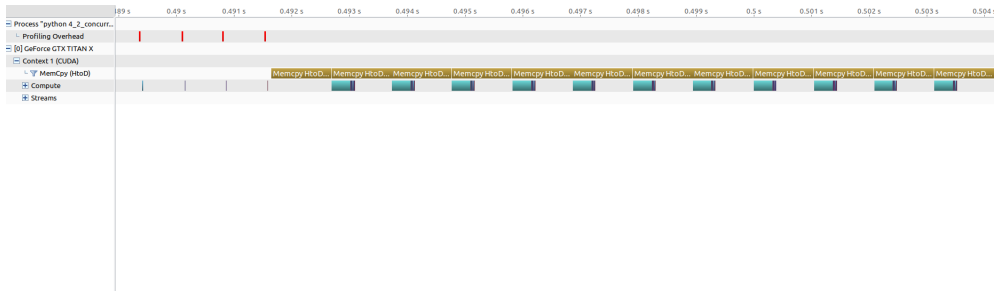
CUDA Streams synchronization: Events

```
data_is_ready = cp.cuda.Event()
gpu_array.set(rf, stream=HtoD_Stream)
data_is_ready.record(stream=HtoD_Stream)
processing_stream.wait_event(data_is_ready)
with processing_stream:
    some_processing(gpu_array)
```



CUDA Streams synchronization: Events

```
data_is_ready = cp.cuda.Event()
gpu_array.set(rf, stream=HtoD_Stream)
data_is_ready.record(stream=HtoD_Stream)
processing_stream.wait_event(data_is_ready)
with processing_stream:
    some_processing(gpu_array)
```



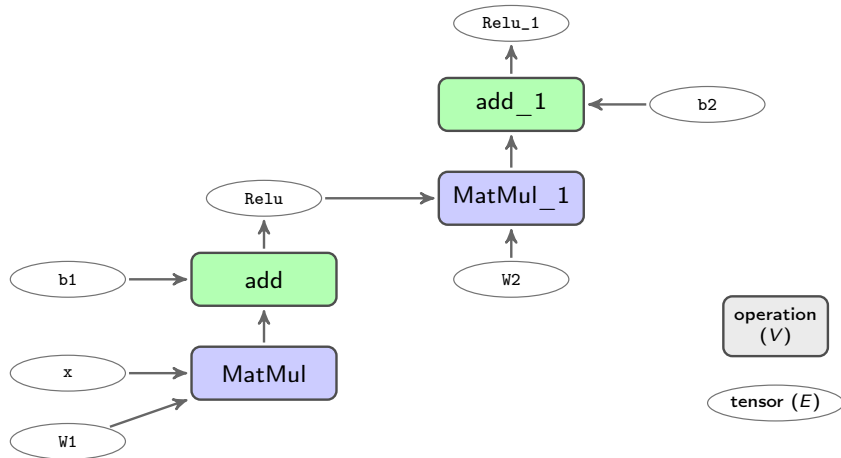
Graph of operations

It's possible to use CUDA kernels and events to implement a graph of operations.

Graph of operations: a Graph $G(V, E)$, where:

- V : **operations**
- E : **dependencies** between operations (input/output arrays)

Graph of operations: example

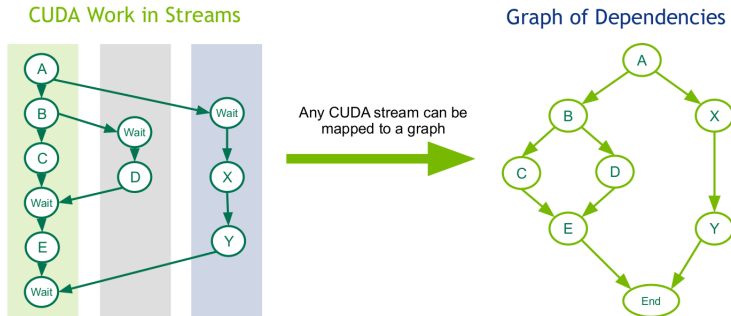


Redrawn after “TensorFlow: introduction to graphs”,
https://www.tensorflow.org/guide/intro_to_graphs

Graph of operations: frameworks

- tensorflow (based on XLA)
- ...
- NVIDIA CUDA Graphs
- NVIDIA CUDA Holoscan SDK

CUDA Graphs

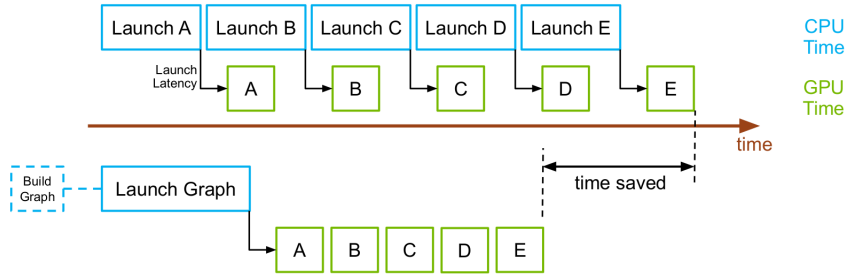


Source: "013 NVIDIA CUDA Graphs",
https://www.olcf.ornl.gov/wp-content/uploads/2021/10/013_CUDA_Graphs.pdf

CUDA Graphs: Execution model



CUDA Graphs: kernel invocation overhead reduction



Source: "013 NVIDIA CUDA Graphs",

https://www.olcf.ornl.gov/wp-content/uploads/2021/10/013_CUDA_Graphs.pdf

NVIDIA Holoscan SDK

NVIDIA Holoscan is the AI sensor processing platform that combines:

- hardware systems for low-latency sensor and
- network connectivity,
- optimized libraries for data processing and AI,
- ...

NVIDIA Holoscan SDK is a development kit that provides:

- C++ and Python API,
- Built-in, common operators (AI inference, etc.),
- examples, a remote repository of operators, performance tools...

Source: "013 NVIDIA CUDA Graphs",

https://www.olcf.ornl.gov/wp-content/uploads/2021/10/013_CUDA_Graphs.pdf

NVIDIA Holoscan SDK: Core concepts

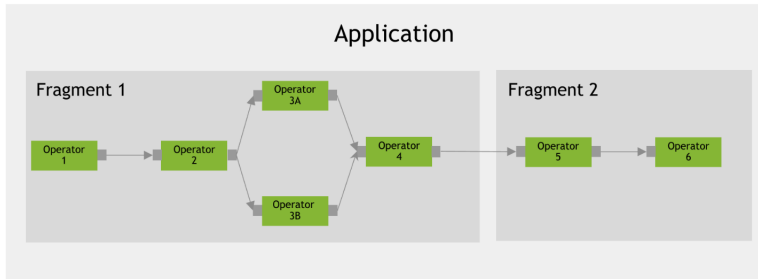
Holoscan SDK implements a custom graph framework: **Graph Execution Framework (GXF)**.

Also provides a native support for:

- GPU Direct RDMA
- TensorRT
- ...

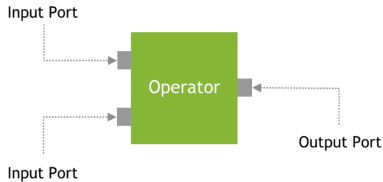
NVIDIA Holoscan SDK: Core concepts

An **Application** is composed of **Fragments**, each of which runs a sub-graph of **Operations**.



NVIDIA Holoscan SDK: Core concepts

Operator is a basic unit of work. An operator receives streaming data at an input port, processes it, and publishes it to one of the output ports.



NVIDIA Holoscan SDK

HoloHub: a central repository to share reusable operators and applications with the Holoscan community.

In preparation:

- us4us us4R $\longleftrightarrow$ NVIDIA Holoscan SDK integration
- GPU RCA image reconstruction using NVIDIA Holoscan SDK (offline processing)

Please make sure to subscribe to the 'github.com/lab4us/gpu-short-course' repository by pressing 'Star' button. This way you will get notification when the above are ready.
Thanks!

Multiple GPUs

GPU ↔ GPU

Peer-to-peer: one GPU accesses the memory of another directly, without the host. `cudaDeviceEnablePeerAccess` + `cudaMemcpyPeerAsync` (HIP: `hipDeviceEnablePeerAccess`, `hipMemcpyPeerAsync`). Over **NVLink** / **xGMI** when available, over PCIe otherwise.

many GPUs

Collective libraries: **NCCL** (NVIDIA), **RCCL** (AMD), or GPU-aware MPI – broadcast, all-reduce, point-to-point, with the topology taken into account.

device → GPU

GPUDirect RDMA (NVIDIA) / **DMA-BUF**, **PeerDirect** (AMD): the acquisition hardware or the NIC writes straight into GPU memory.

Isolating GPU resources

Can one stage (e.g. beamforming) get a *guaranteed* slice of the GPU?

NVIDIA

- **MIG**: hardware partitions of SMs, L2 and memory (A100 / H100 class), up to 7 instances – real isolation,
- **MPS**: several processes on one GPU, `CUDA_MPS_ACTIVE_THREAD_PERCENTAGE` and a memory limit per client – a soft limit,
- **green contexts** (CUDA 12.4+): SM partitions inside one process,
- stream priorities: a scheduling hint only.

Partitioning is a *system* decision (driver / admin), not something a kernel can ask for at run time.

AMD

- **compute partitioning** (MI300/MI350): SPX, DPX, TPX, QPX, CPX modes – the XCDs are exposed as separate devices,
- **memory partitioning**: NPS1 ... NPS4,
- **CU masks**: `hipExtStreamCreateWithCUMask` restricts a stream to a set of compute units,
- SR-IOV for virtual machines.